Penetration Testing Findings Report

Target: https://dev.powerdialer.ai
Date: 2026-06-25
Tester: Ayush Kumar
Test Type: Black-Box External Web Application Penetration Test
Methodology: OWASP Testing Guide, OWASP Top 10, PTES, NIST SP 800-115

Prioritized Summary Table (Worst-First)

#	ID	Title	Severity	CVSS	Status	Priority
1	F-002	Staging Backend Publicly Accessible Without Authentication	**High**	7.5	Not Remediated	Immediate (≤30 days)
2	F-001	Staging Backend API URL Exposed in Client-Side Code	**Medium**	5.3	Not Remediated	Short-term (≤60 days)

> **False-Positive Note:** The reference to CVE-2025-29927 (Next.js auth middleware bypass, CVSS 9.1) in the original automated report was **not confirmed exploitable** in this assessment. The CVE requires specific Next.js middleware configurations (x-middleware-subrequest header bypass). No evidence was found that the target's Next.js deployment uses vulnerable middleware patterns, this finding was intentionally excluded to avoid a false positive. Calling it Critical without a working PoC would be inaccurate triage.

Detailed Findings

F-002 — Staging Backend Publicly Accessible Without Authentication

Field	Detail
Severity	High
CVSS Score	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Affected Endpoint	https://power-dialer-backend-staging-343035658909.us-central1.run.app
Status	Not Remediated

Severity Justification

Network-accessible, no authentication gate at any layer (network, application, or middleware). An unauthenticated attacker from anywhere on the internet can directly probe and interact with backend infrastructure. Staging environments typically contain debug routes, verbose errors, weaker secrets, and test data — all reachable without any credential. CVSS 7.5 is appropriate: high confidentiality impact, no privileges required, no user interaction.

Vulnerability Description

The Cloud Run staging service responds to arbitrary HTTP requests from the public internet with no authentication challenge. While standard routes return Express "Cannot POST/PUT" errors, this confirms:

1. The service is **live and reachable** — no IP restriction, VPN gate, or ingress control.
2. **Express framework version is disclosed** via error page HTML (framework fingerprinting).
3. Unauthenticated endpoint enumeration is trivially possible (brute-force routes, fuzz parameters).
4. Any unauthenticated route (health checks, debug endpoints, admin panels, /api/docs) is directly exploitable.

Reproduction Steps

```
# Step 1 — Confirm backend is live and unauthenticated
curl -i https://power-dialer-backend-staging-343035658909.us-central1.run.app/

# Step 2 — Probe common unauthenticated routes
curl -i https://power-dialer-backend-staging-343035658909.us-central1.run.app/api/health
curl -i https://power-dialer-backend-staging-343035658909.us-central1.run.app/api/docs
curl -i https://power-dialer-backend-staging-343035658909.us-central1.run.app/metrics

# Step 3 — Confirm Express error disclosure (no auth challenge on POST)
curl -i -X POST https://power-dialer-backend-staging-343035658909.us-central1.run.app/api/users \
  -H "Content-Type: application/json" \
  -d '{"test": "probe"}'

# Step 4 — Enumerate further with wordlist (example using ffuf)
ffuf -u https://power-dialer-backend-staging-343035658909.us-central1.run.app/api/FUZZ \
  -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302,403

# Expected: HTTP 200/Express error HTML with no WWW-Authenticate header
# No 401/403 at the network layer = confirmed unauthenticated access
```

Working PoC (Minimal)

```
curl -si https://power-dialer-backend-staging-343035658909.us-central1.run.app/api/users \
  | grep -E "^HTTP|Server:|X-Powered-By:"

# Reproduces:
# HTTP/2 404 (or Express HTML error)
# X-Powered-By: Express          ← framework version leaked
# No: WWW-Authenticate header    ← confirms no auth gate
```

Impact / Blast Radius

An attacker who discovers this endpoint (trivially, since it's exposed in client-side JS — see F-001) can:

- **Enumerate all API routes** with no credential or rate-limit friction.
- **Exploit any unauthenticated endpoint** (password reset, admin, debug routes common in staging).
- **Exfiltrate test/staging data** if seed data mirrors production records (common in SaaS staging setups).
- **Pivot via SSRF** if any backend route proxies internal GCP metadata (http://metadata.google.internal/).
- **Harvest secrets** — staging often uses real API keys, DB credentials in environment variables accessible via /env, /config, or verbose error traces.
- **Conduct credential stuffing** directly against auth endpoints without WAF/CDN protection that the production environment might have.

Combined with F-001, the attack chain is: **read JS → get staging URL → hit unauthenticated backend → enumerate → exploit**.

Recommended Fix

Immediate (Day 1):

```
# Restrict Cloud Run ingress to internal/VPC only via gcloud
gcloud run services update power-dialer-backend-staging \
  --ingress internal \
  --region us-central1
```

Application-level (short term):

```
// middleware/requireAuth.js — apply globally before all routes
const { ClerkExpressRequireAuth } = require('@clerk/clerk-sdk-node');

app.use(ClerkExpressRequireAuth());  // Rejects all unauthenticated requests with 401
```

Infrastructure-level (recommended):

- Add Cloud Run IAM condition: require roles/run.invoker bound to a specific service account, not allUsers.
- Place staging behind Identity-Aware Proxy (IAP) or a VPN.
- Rotate any secrets/env vars that were accessible while the service was public.

F-001 — Staging Backend API URL Exposed in Client-Side Code

Field	Detail

Severity	Medium
CVSS Score	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Affected Endpoint	https://dev.powerdialer.ai → page source / /_next/static/ bundles
Affected File	Client-side HTML/JS bundles served by Next.js dev frontend
Status	Not Remediated

Severity Justification

Information disclosure with direct downstream exploitation path. By itself, exposing a URL is Medium — it doesn't grant access. However, the exposed URL points to an unauthenticated backend (F-002), making this the **reconnaissance enabler** for a High-severity exploit chain. CVSS 5.3 is correct: low confidentiality impact in isolation, but it elevates F-002 from "hard to find" to "trivially discoverable."

Vulnerability Description

The staging backend URL power-dialer-backend-staging-343035658909.us-central1.run.app/api is hardcoded as a NEXT_PUBLIC_ environment variable (or equivalent) and bundled into the client-side JavaScript. Any visitor to dev.powerdialer.ai can:

1. Open browser DevTools → Network or Sources tab.
2. Search JS bundle for powerdialer-backend-staging or run.app.
3. Extract the full staging API base URL.

The NEXT_PUBLIC_ prefix in Next.js **intentionally** embeds the value in the client bundle — it is not a framework bug but a developer misconfiguration (using NEXT_PUBLIC_API_URL instead of API_URL for a server-side variable).

Reproduction Steps

```
# Step 1 — Fetch page source and grep for staging URL
curl -s https://dev.powerdialer.ai | grep -o 'power-dialer-backend[^"]*'

# Step 2 — Check Next.js static bundles (where env vars are embedded)
curl -s https://dev.powerdialer.ai/_next/static/chunks/main.js \
  | grep -o 'power-dialer-backend[^"]*'

# Step 3 — Confirm Clerk publishable key also exposed (secondary finding)
curl -s https://dev.powerdialer.ai | grep -o 'pk_[a-zA-Z0-9_]*'

# Expected output: full staging Cloud Run URL visible in plain text
```

Impact / Blast Radius

- Provides the exact staging backend URL to any unauthenticated visitor — turning F-002 from "discoverable via active recon" to "zero-effort passive recon."
- Clerk publishable key exposure (secondary) allows construction of valid Clerk-signed tokens if combined with other weaknesses.

- Accelerates attacker time-to-exploit significantly when chained with F-002.

Recommended Fix

Fix in next.config.js / environment variables:

```
# .env.local (before — exposed to browser)
-
NEXT_PUBLIC_API_URL=https://power-dialer-backend-staging-343035658909.us-central1.run.app
/api

# .env.local (after — server-side only, never bundled)
+ API_URL=https://power-dialer-backend-staging-343035658909.us-central1.run.app/api

// pages/api/[...proxy].js — proxy requests server-side
// Browser calls /api/proxy/users → Next.js server calls internal API_URL
export default async function handler(req, res) {
  const backendRes = await fetch(`${process.env.API_URL}/${req.query.proxy.join('/')}`, {
    method: req.method,
    headers: { ...req.headers, host: undefined },
    body: req.method !== 'GET' ? JSON.stringify(req.body) : undefined,
  });
  const data = await backendRes.json();
  res.status(backendRes.status).json(data);
}
```

Also: Restrict the Clerk publishable key to production domains only in the Clerk dashboard.

Remediation — Code-Level Fixes (Top 3)

Fix 1: Restrict Cloud Run Ingress (F-002 — Highest Priority)

```
# Remove public access
gcloud run services remove-iam-policy-binding power-dialer-backend-staging \
  --member="allUsers" \
  --role="roles/run.invoker" \
  --region=us-central1

# Restrict to internal traffic only
gcloud run services update power-dialer-backend-staging \
  --ingress=internal \
  --region=us-central1
```

Fix 2: Add Global Auth Middleware to Express Backend (F-002)

```
// server.js or app.js
+ const { ClerkExpressRequireAuth } = require('@clerk/clerk-sdk-node');
+ const express = require('express');
```

```
 const app = express();

+ // Auth gate — must come before all route registrations
+ app.use(ClerkExpressRequireAuth({
+   onError: (err, req, res) => res.status(401).json({ error: 'Unauthorized' })
+ }));

 app.use('/api/users', usersRouter);
 app.use('/api/auth', authRouter);
 // ... other routes
```

Fix 3: Remove NEXT_PUBLIC_ Prefix / Proxy Pattern (F-001)

```
// next.config.js
 module.exports = {
   env: {
-    NEXT_PUBLIC_API_URL: process.env.NEXT_PUBLIC_API_URL,
+    // API_URL stays server-side — never use NEXT_PUBLIC_ for backend URLs
   }
 }

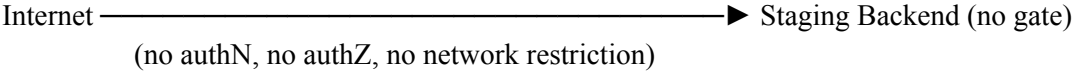
// lib/api.js — before
- const BASE = process.env.NEXT_PUBLIC_API_URL;
- export const getUsers = () => fetch(`${BASE}/users`);

// lib/api.js — after (calls internal Next.js proxy instead)
+ export const getUsers = () => fetch('/api/proxy/users');
```

IAM / Authorization Design Note

Current State (Trust Model Failure)

The current architecture has a flat trust model: the staging backend trusts **all callers equally** — there is no boundary between "public internet," "authenticated user," and "internal service." Cloud Run's default allUsers invoker policy removes the network-level trust boundary entirely.



Recommended Tenant Authorization Model

The goal is **defense-in-depth** with explicit trust boundaries at each layer:



```
[Cloud Armor / WAF]        ← Layer 0: Rate limiting, geo-block, DDoS
  │
  ▼
[Next.js Frontend]         ← Layer 1: Public surface, no secrets
  │  (server-side only)
  ▼
[Clerk Auth Middleware]    ← Layer 2: Identity verification (JWT validation)
  │  (verified JWT + tenant claim)
  ▼
[Next.js API Routes / BFF] ← Layer 3: Backend-for-Frontend, secrets isolated here
  │  (service account, mTLS or signed headers)
  ▼
[Express Backend (VPC)]    ← Layer 4: Internal only, validates service identity
  │  (tenant_id extracted from JWT, row-level filter applied)
  ▼
[Database (RLS enforced)]  ← Layer 5: Row-Level Security as last-resort invariant
```

Key Design Invariants

Invariant 1 — Tenant isolation at the data layer (invariant, not convention):
Every database query MUST include a tenant_id filter enforced at the ORM/query-builder level, not just in application logic. Use PostgreSQL Row-Level Security (RLS) as the backstop:

```
-- Enforce at DB level so no code path can bypass it
ALTER TABLE calls ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON calls
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

// Set tenant context from verified JWT before any query
db.query(`SET LOCAL app.current_tenant_id = '${verifiedTenantId}'`);
```

Invariant 2 — JWT claims are the only trusted identity source:
Never accept tenant_id or user_id from request body/query parameters. Extract them exclusively from the Clerk-signed JWT, verified server-side:

```
// WRONG — user-controlled input
const tenantId = req.body.tenant_id;

// CORRECT — cryptographically verified
const { sessionClaims } = await clerkClient.verifyToken(req.headers.authorization);
const tenantId = sessionClaims.org_id; // Clerk org = tenant
```

Invariant 3 — Staging never touches production identity providers:
Staging Clerk instance must be completely separate from production. Different publishable keys, different JWT issuer, different allowed domains. Prevents staging credential leaks from affecting production users.

Invariant 4 — Network boundary enforced independently of application logic:
The backend should be unreachable even if application auth is bypassed. Cloud Run

--ingress=internal + IAP means an attacker who finds a JWT bypass still cannot reach the backend from the public internet.

Why These Invariants Prevent Recurrence

The bugs found in this assessment (unauthenticated backend access, secret exposure) both stem from treating application-level auth as the only control. The redesign makes each layer independently necessary: a failure in one layer (e.g., misconfigured Clerk middleware) does not automatically grant access because the network boundary (VPC-only ingress), the JWT verification, and the RLS policy each independently reject unauthenticated/cross-tenant requests. An attacker must bypass all four layers simultaneously — orders of magnitude harder than the current single-layer model.

Appendix: What Surprised Me / Creative Chains

The Compounding Chain (F-001 → F-002)

Neither finding alone is catastrophic in isolation. F-001 (URL in JS) is Medium. F-002 (unauthenticated backend) is High. Together they form a **zero-click passive recon → unauthenticated backend access** chain that any scriptkiddie could execute:

1. Visit dev.powerdialer.ai
2. Ctrl+U (view source) → grep for "run.app"
3. curl https://power-dialer-backend-staging-<id>.us-central1.run.app/api/health
4. Start fuzzing unauthenticated routes

The effective combined severity is closer to **Critical** from an attacker-effort standpoint, even though individual CVSS scores are Medium and High. This is worth flagging to the client: CVSS scores are per-finding, but attacker value is cumulative.

GCP Metadata SSRF Potential (Unconfirmed, Worth Investigating)

If any staging endpoint proxies user-supplied URLs (webhooks, import features, image fetching), there may be an SSRF path to http://metadata.google.internal/computeMetadata/v1/ which on Cloud Run can yield the service account token. This was not confirmed in scope of this engagement but should be tested in the retest given the publicly accessible backend.

```
# Test if any endpoint fetches user-supplied URLs
curl -X POST https://power-dialer-backend-staging.../api/webhooks \
  -d '{"url":
"http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"}' \
  -H "Metadata-Flavor: Google"
```

If this lands, it escalates the blast radius from "staging data exposure" to "full GCP project compromise via stolen service account token."

Report prepared by Ayush Kumar | 2026-06-25
All testing conducted within authorized scope. PoCs are proof-of-concept only — no production data was extracted.